

# CS-468

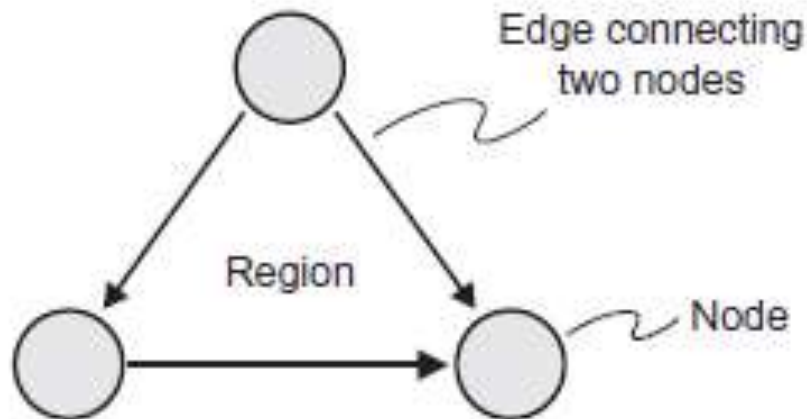
# Software Quality Assurance & Testing

Week 11

Session 1

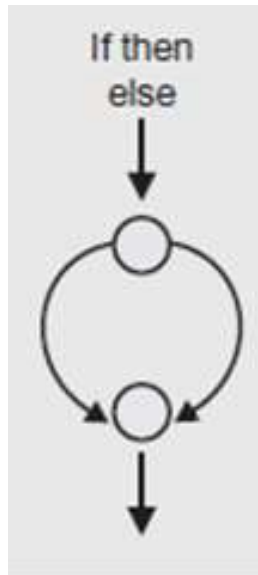
# Control Flow Graph

- A control flow graph provides a method of representing the decision points and the flow of control within a piece of code, so it is just like a flow chart except that it only shows decisions.
- A control flow graph is produced by looking only at the statements affecting the flow of control.
- The graph itself is made up of two symbols: nodes and edges. A node represents any point where the flow of control can be modified (i.e. decision points), or the points where a control structure returns to the main flow (e.g. END WHILE or ENDIF).
- An edge is a line connecting any two nodes. The closed area contained within a collection of nodes and edges, as shown in the diagram, is known as a region.

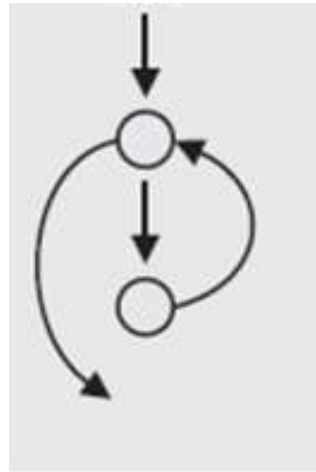


# Control Flow Subgraphs

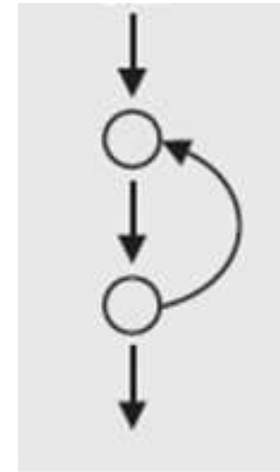
- We can draw 'sub graphs' to represent individual structures.
- For a flow graph the representation of sequence is just a straight line, since there is no decision to cause any branching.



Pre- Test Loop  
While & For



Post- Test Loop  
Do- While



- The sub graphs show what the control flow graph would look like for the program structures we are already familiar with.
- Any chunk of code can be represented by using these subgraphs.

# Drawing a Control flow Graph

- The steps are as follows:
  - Analyse the component to identify all control structures, i.e. all statements that can modify the flow of control, ignoring all sequential statements.
  - Add a node for any decision statement.
  - Expand the node by substituting the appropriate sub graph representing the structure at the decision point.

# Example

- Here is a simple program for calculating the mean and maximum of three integers.
- It has some non-executable statements (those before the Begin and those after the Begin that are actually blank lines) that we will have to take account of when we come to count the number of executable statements later.
- The line numbering makes it a little easier to do the counting.
- The program does not recognise if two of the numbers are the same value.

```
1  Program MaxandMean
2
3  A, B, C, Maximum: Integer
4  Mean: Real
5
6  Begin
7
8  Read A
9  Read B
10 Read C
11 Mean = (A + B + C)/3
12
13 If A > B
14 Then
15     If A > C
16     Then
17         Maximum = A
18     Else
19         Maximum = C
20     Endif
21 Else
22     If B > C
23     Then
24         Maximum = B
25     Else
26         Maximum = C
27     Endif
28 Endif
29
30 Print ("Mean of A, B and C is ", Mean)
31 Print ("Maximum of A, B, C is ", Maximum)
32
33 End
```

## ... Example - Solution

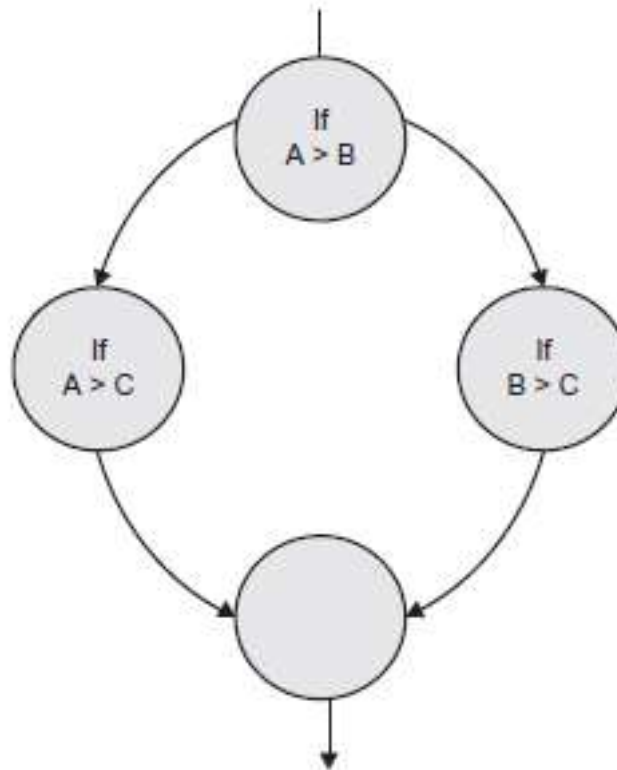
- Step 1 breaks the code into statements and **identifies the control structures**, ignoring the sequential statements, in order to identify the decision points; these are highlighted below.

```
1  Program MaxandMean
2
3  A, B, C, Maximum: Integer
4  Mean: Real
5
6  Begin
7
8  Read A
9  Read B
10 Read C
11 Mean = (A + B + C)/3
12
13 If A > B
14 Then
15     If A > C
16     Then
17         Maximum = A
18     Else
19         Maximum = C
20     Endif
21 Else
22     If B > C
23     Then
24         Maximum = B
25     Else
26         Maximum = C
27     Endif
28 Endif
29
30 Print ("Mean of A, B and C is ", Mean)
31 Print ("Maximum of A, B, C is ", Maximum)
32
33 End
```

## ... Example – Solution ...

- Step 2 **adds a node for each branching or decision statement.**

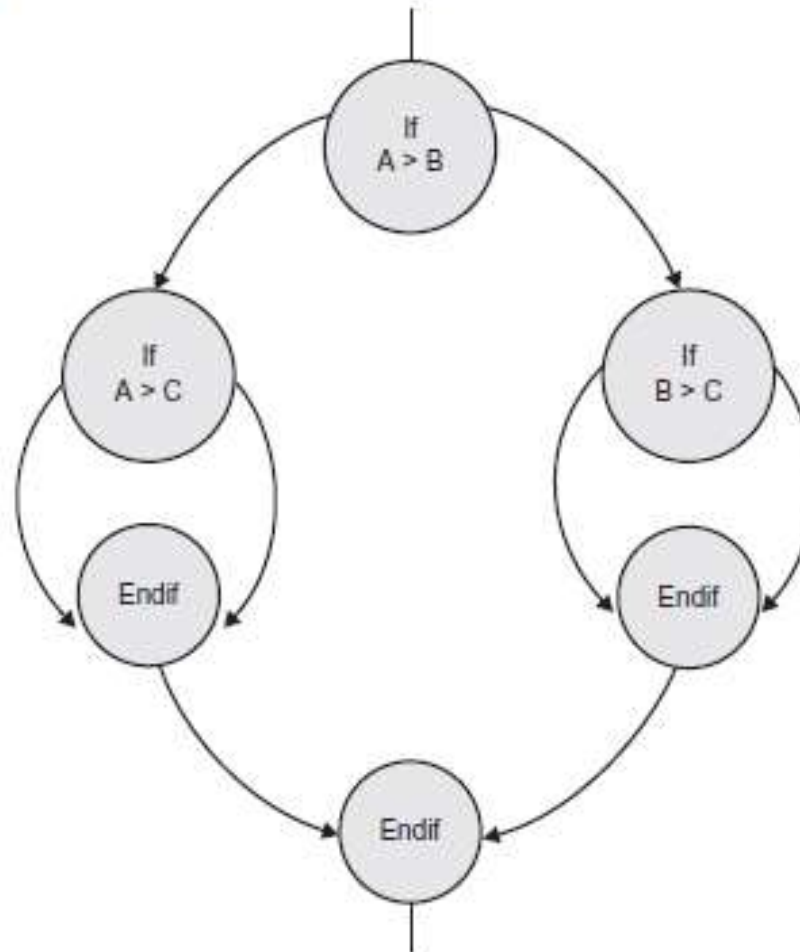
Control flow graph showing subgraphs as nodes



## ... Example – Solution

- Step 3 **expands the nodes by substituting the appropriate subgraphs**

Control flow graph with subgraphs expanded

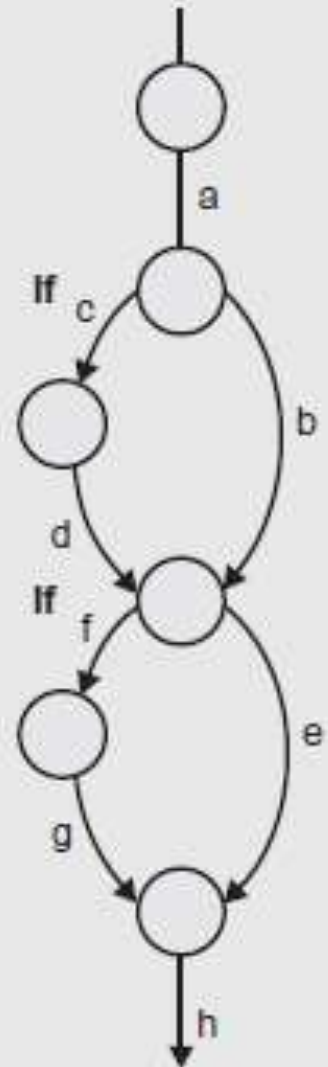




# Hybrid Flow Graph

- Flow charts, control flow graphs and hybrid flow graphs all show essentially the same information, but sometimes one format is more helpful than another.
- We have identified the hybrid flow graph as a useful combination of the control flow graph and the control flow chart.
- To make it even more useful we can add to it labels to indicate the paths that a program can follow through the code.
- All we need to do is to label each edge; paths are then made up from sequences of the labels, such as **'abeh'**, which make up a path through the code.

Paths through the hybrid flow graph



# Example

- Here is a program. How many test cases will you need to achieve 100 percent statement coverage and what will the test cases be?

```
1  Program BestInterest
2  Interest, Base Rate, Balance: Real
3
4  Begin
5  Base Rate = 0.035
6  Interest = Base Rate
7
8  Read (Balance)
9  If Balance > 1000
10 Then
11     Interest = Interest + 0.005
12     If Balance < 10000
13     Then
14         Interest = Interest + 0.005
15     Else
16         Interest = Interest + 0.010
17     Endif
18 Endif
19
20 Balance = Balance × (1 + Interest)
21
22 End
```

# Example - Solution

- It is clear from the flow graph that the left-hand side (Balance below £1,000) need not be exercised, but there are two alternative paths (Balance between £1,000 and £10,000 and Balance > £10,000) that need to be exercised.
- So we need two test cases for 100 per cent statement coverage and Balance = £5,000, Balance = £20,000 will be suitable test cases.
- Alternatively we can aim to follow the paths 'abcegh' and 'abdfgh' marked on the flow graph.
- How many test cases do we need to do that?
- We can do this with one test case to set the initial balance value to a value between £1,000 and £10,000 (to follow 'abcegh') and one test case to set the initial balance to something higher than £10,000, say £12,000 (to follow path 'abdfgh').
- So we need two test cases to achieve 100 per cent statement coverage in this case.**

Paths through the hybrid flow graph

